Two simple heptagons sharing no vertex meet in at most 38 points: what the machine checked, and what it did not

An audit document for the Lean development,
written for readers with or without Lean experience

Revised 4 October 2026 (first version 24 September 2026)

In one paragraph. A Lean 4 development in this repository contains a kernel-checked theorem: *if two simple heptagons in the plane share no vertex and their boundaries meet in finitely many points, then the boundaries meet in at most* 38 *points; and* 38 *is attained.* This includes every pair in general position. The recorded build of the theorem shows no `sorry`, no axiom declared by this project, and no `native_decide`. The theorem is **not** by itself the statement $f(7,7) = 38$, which ranges also over pairs that share vertices: that step uses the vertex-coincidence reduction of §9.1 of Ackerman–Keszegh–Rote, which is not formalized here and whose applicability to these exact definitions this project has not fully checked (§8). This document sets out the formal statement, the evidence for it, and the limits of that evidence.

0. What the evidence does and does not establish

Kernel checking. A Lean file is a set of instructions for constructing proof objects, which Lean's kernel checks against a fixed set of inference rules. If the development builds, the kernel has accepted a proof of each theorem's formal statement, under its stated hypotheses and Lean's foundational axioms, relative to the correctness of Lean itself and of the Mathlib definitions the statements use. This development is 1,779 `.lean` files and about 1.6 million lines; most files are generated case enumerations, and the proofs of the headline theorems are long generated propagation chains. They are not written to be read line by line, and reading them is not the usual way such a development is reviewed. Kernel checking does not establish that a formal statement says what its author intends, and it does not discharge the statement's hypotheses.

The statement and definitions. What a reviewer must read is the statement and the definitions it uses. If `points9` denoted the wrong set, or `SimpleClosed` encoded more than simplicity, a correct kernel-checked proof would establish something other than the intended result. The statement surface quoted here is 16 declarations, and the seven that carry the content are printed in §2 beside a reading in ordinary mathematics. Other risks remain alongside the definitions: a hypothesis stronger than intended (addressed for satisfiability by §7), a mismatch between the code quoted here and the code that was built (addressed partly by the drift check of §2), and the external reduction (§8).

The axiom check. Lean's `#print axioms` lists the axioms a declaration's proof depends on, transitively. Three kinds of dependency are relevant here, and the repository's axiom checker looks for each:

- `sorry`, a placeholder for a missing proof. A file containing it still compiles, with a warning, and the build exits 0. It appears in the axiom list as `sorryAx`.
- A declared `axiom`, a statement asserted without proof. It appears by name.
- `native_decide`, which evaluates a goal with compiled code rather than in the kernel, enlarging the trusted code base to the compiler. It appears as `Lean.ofReduceBool`.

The expected output for the headline theorems is

```
[propext, Classical.choice, Quot.sound],
```

Lean's standard foundational axioms, which Mathlib also uses. The check is evidence about the declarations whose axiom lists are printed and parsed. It does not show that a statement has the intended meaning, it does not discharge hypotheses, and it does not detect errors in Lean itself.

Recorded builds. The repository's GitHub Actions workflow builds the headline target from a fresh checkout on a GitHub-hosted runner, runs the axiom check, and runs the group-A tests of §4. Two runs have passed (§6). A run records that the committed workflow passed at one revision. The repository owner writes the workflow, so the run shows what that workflow checked, not an independent party's choice of checks. It is not a mathematical review.

What a reader still has to judge.

1. **The toolchain:** that Lean's kernel, and the Mathlib definitions used in the statement, are correct. This is assumed by every Lean formalization.
2. **The statement:** that the formal statement in §2 expresses the intended theorem. The table in §2 supports this check; it is easier for a reader who knows Lean.
3. **The connection between this document and the code:** that what is quoted here is what was built (§§2 and 5).
4. **The external reduction:** AKR §9.1, and whether it applies to these definitions (§8).

1. The claim, in ordinary mathematics

Proved. Let P and Q be simple closed heptagons in the plane — seven vertices each, sides $[P_i, P_{i+1}]$ and $[Q_j, Q_{j+1}]$ for $i, j \in \mathbb{Z}/7$, each polygon a closed curve that does not cross itself. Suppose $\partial P \cap \partial Q$ is finite and no vertex of P is equal to a vertex of Q. Then $\#(\partial P \cap \partial Q) \leq 38$, and there is such a pair for which the count is exactly 38; so 38 is the maximum over such pairs. Every pair in general position satisfies these hypotheses.

Hypotheses, not conclusions. Finiteness and "no shared vertex" are hypotheses of the theorem, written out explicitly. That every pair of simple heptagons with finite intersection can be replaced by one with no shared vertex, again simple and with no fewer intersection points, is a separate *reduction*: it is the subject of §9.1 of Ackerman, Keszegh and Rote, and it is not formalized here (§8).

Two remarks on what is *not* assumed, because these are the places where a formal statement is most easily narrower than intended.

No general position. Three collinear vertices are allowed. A straight vertex — where two consecutive sides are collinear — is allowed. A vertex of one polygon lying in the *interior* of a side of the

other is allowed, and is the case AKR §9.2 removes by a perturbation argument; here it is proved, not assumed away. The only degeneracy excluded is vertex-equals-vertex.

Points, not crossings. The quantity bounded is the cardinality of a set of points of the plane. A tangential touch that does not cross counts as one point; a point where three sides concur counts once, not twice. This is the harder thing to bound and the thing the question actually asks.

2. The statement, beside the mathematics

This is the part to read. The left column is the claim in ordinary mathematical English; the right column is the Lean source, quoted verbatim by `docs/audit/extract_lean.py` from the files and line numbers shown. **Your task is to decide whether the two columns say the same thing.** Nothing else in this document depends on your knowing any Lean: the notation you need is $\forall$, $\wedge$ (and), $\to$ (implies), $\cup$, $\cap$, `Fin 7` $= \mathbb{Z}/7$ with wraparound on `+`, and `segment ℝ a b` $= [a, b]$, the closed segment.

the mathematics	the Lean, verbatim

(1) P is a closed heptagon that does not cross itself: no side is a point, non-adjacent sides are disjoint, and two consecutive sides meet exactly in the vertex they share.	`def SimpleClosed (W : Fin 7 → ℝ × ℝ) : Prop :=`
	`  (∀ i : Fin 7, W i ≠ W (i + 1)) ∧`
	`  (∀ i j : Fin 7, ¬ LowerBound.Adjacent i j →`
	`  Disjoint (segment ℝ (W i) (W (i + 1))) (segment ℝ (W j) (W (j + 1)))) ∧`
	`  (∀ i : Fin 7,`
	`  segment ℝ (W i) (W (i + 1)) ∩ segment ℝ (W (i + 1)) (W (i + 2))`
	`  = {W (i + 1)})`
	`lean/AKR9.lean:71`
	
	Note. This is simplicity *as a curve* and nothing more. Three collinear vertices are deliberately allowed — a straight vertex is not excluded here. `Adjacent` is cyclic adjacency on $\mathbb{Z}/7$.

(2) The object under study is an ordered pair (P, Q) of such heptagons in the plane. Nothing else is assumed about it.	`structure Pair9 where`
	`  /-- The fourteen vertices: `P` is `0..6`, `Q` is `7..13`. -/`
	`  V : Fin 14 → ℝ × ℝ`
	`  simpleP : SimpleClosed (fun i : Fin 7 => V (pP i))`
	`  simpleQ : SimpleClosed (fun i : Fin 7 => V (pQ i))`
	`lean/AKR9.lean:81`
	
	Note. The fourteen vertices are one function `V`; `pP i` $= i$ and `pQ i` $= 7 + i$ split them. There is no general-position field, and no non-degeneracy field. The two `simple` fields are the whole hypothesis.

(3) $\partial P = \bigcup_{i \in \mathbb{Z}/7} [P_i, P_{i+1}]$, the union of the seven closed sides; likewise ∂Q.	`def bdryP9 (R : Pair9) : Set (ℝ × ℝ) := ⋃ i : Fin 7, sideP9 R i`
	`lean/AKR9.lean:115`
	
	Note. `segment ℝ a b` is Mathlib's *closed* segment $\{(1-t)a + tb : 0 \le t \le 1\}$, so endpoints are included. `sideP9` is that segment; `bdryQ9` is the same with `pQ`.

(4) The set being counted is $\partial P \cap \partial Q$: the points of the plane lying on both boundaries.	`def points9 (R : Pair9) : Set (ℝ × ℝ) := bdryP9 R ∩ bdryQ9 R`
	`lean/AKR9.lean:119`
	
	Note. A set of points, not a list of crossings and not a multiset. A point where three sides concur is one element, not two.

(5) *Hypothesis:* no vertex of P equals a vertex of Q. (Removing shared vertices is the reduction of AKR §9.1, not formalized here; see §8.)

```
def attainedCoincFree : Set ℕ :=
  { n | ∃ R : Pair9, (points9 R).Finite ∧
      (∀ v j : Fin 7, R.V (pP v) ≠ R.V (pQ j)) ∧ (points9 R).ncard = n }
```

lean/Pair9Shift.lean:4314

Note. The middle clause is the assumption, written out. It forbids *equality* of vertices only — a vertex lying in the interior of the other polygon's side is *not* excluded, and is handled by the proof.

---

(6) If $\partial P \cap \partial Q$ is finite and P, Q share no vertex, then $\#(\partial P \cap \partial Q) \leq 38$.

```
theorem points9_le_38_of_coincFree (R : Pair9) (hfin : (points9 R).Finite)
    (hcoinc : ∀ v j : Fin 7, R.V (pP v) ≠ R.V (pQ j)) :
    (points9 R).ncard ≤ 38 := by
  obtain ⟨Rend, hD, hE, hb⟩ := exists_chain_deg_free _ R hfin hcoinc rfl
  exact points9_le_38_of_joint R Rend hfin hb hD hE
```

lean/Pair9Shift.lean:4223

Note. `ncard` is the cardinality of a set as a natural number. `hfin` is the finiteness hypothesis; without it the count is not a number at all (see the last row of §3).

---

(7) 38 is attained, and is the largest attained value: it is the maximum, not merely an upper bound.

```
theorem isGreatest_38_of_coincFree : IsGreatest attainedCoincFree 38 := by
  refine ⟨⟨AKR9.certPair9, certPair9_finite, certPair9_coincFree,
    AKR9.certPair9_ncard⟩, ?_⟩
  rintro n ⟨R, hfin, hco, rfl⟩
  exact points9_le_38_of_coincFree R hfin hco
```

lean/Pair9Shift.lean:4324

Note. `IsGreatest S a` unfolds to $a \in S \wedge \forall b \in S,\ b \leq a$. The membership half is the explicit certificate of §3; the bound half is the row above.

The drift check. `extract_lean.py --check` recomputes the SHA-256 of each of the 16 quoted declarations and compares it against the lockfile written when this document was generated. If one of them has been renamed, removed, or altered by one character, it exits 1 and names it. It is part of the documented procedure for building this PDF. It covers only the quoted text: a change to a definition those declarations use but that is not quoted here — for example `LowerBound.Adjacent`, or Mathlib's `segment` — would not be detected by it.

One thing this table is not. The repository contains a *second* chain that also ends at the number 38 — `Heptagon.MainTheorem.isGreatest_38`, over integer-coordinate combinatorial configurations — and §5 prints its axiom line alongside these. It is not the theorem above: it carries three explicit hypotheses that that route does not itself prove, and §5 sets them out. The six axiom lines in §5 are six statements with different hypotheses, not six versions of one result. The claim this document makes is the one in the table above, and nothing in §§1–4 or 7 depends on the other chain.

3. The gallery: hand-countable pictures beside the computed count

The definitions in §2 are set-theoretic. This section checks that they compute what you would count by eye. `docs/audit/gallery.py` transcribes the same predicates into Python — `orientR`, `SimpleClosed`, `Adjacent`, `points9`, each annotated with the Lean file and line it came from — draws 10 configurations, and circles and numbers every point of $\partial P \cap \partial Q$. All arithmetic is exact rational arithmetic (`fractions.Fraction`); no floating point is used for any geometric decision. The tool self-tests 33 properties on startup and aborts if any fails, including a sweep confirming its segment-intersection routine agrees with the textbook orientation test on all 6,561 segment pairs over a 3×3 grid.

Lead with the certificate. Figure 1 is `certificate_38`: the configuration read from the file `lower_bound_38.json`, the same object the Lean proof uses as its witness. Its 38 intersection points are circled and numbered 1 through 38, so a reader can count them. The kernel-checked counterpart is `AKR9.certPair9_ncard`, which computes the same count from the same coordinates.

configuration	both simple?	**computed**	you should get	degeneracies present
`certificate_38`	yes	38	38	—
`easy_lens_2`	yes	2	2	—
`easy_bars_cross_4`	yes	4	4	—
`easy_slotted_and_bar_8`	yes	8	8	—
`easy_turned_pair_14`	yes	14	14	—
`awkward_vertex_in_edge_interior`	yes	2	2	no vertex on other: **fails**; not general position
`awkward_three_collinear_in_P`	yes	4	4	not general position
`awkward_shared_vertex`	yes	2	2	no vertex on other: **fails**; not general position
`awkward_tangential_touch`	yes	1	1	no vertex on other: **fails**; not general position
`awkward_overlapping_sides_infinite`	yes	INFINITE	infinite	no vertex on other: **fails**; not general position

The five `easy_*` rows are sanity checks with obvious answers (two convex heptagons overlapping in a lens: 2; two crossing bars: 4; a convex heptagon and a turned copy: 14, which is 2 per side and the maximum for two convex heptagons). The five `awkward_*` rows are the ones that matter:

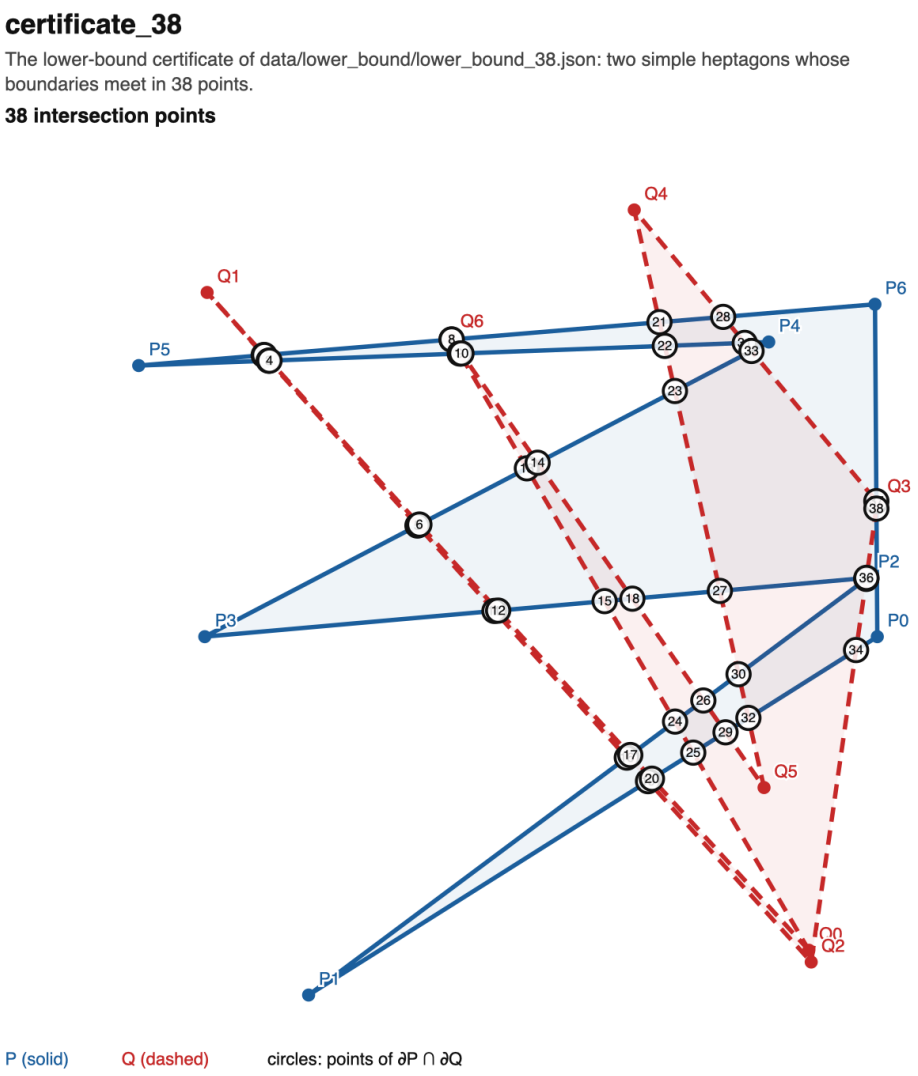
- `awkward_vertex_in_edge_interior` — a vertex of Q lies exactly inside a side of P. This is the degeneracy AKR §9.2 removes and this project proves; it is *not* excluded by the theorem's hypotheses, and it is counted correctly.
- `awkward_three_collinear_in_P` — P_0, P_1, P_2 are collinear, so P's base is one straight line split into two sides. `SimpleClosed` accepts this, deliberately. Any definition that rejected it would be smuggling in general position.
- `awkward_shared_vertex` — $P_0 = Q_0$. This is exactly the case the theorem assumes away. It is drawn here so you can see what the hypothesis excludes.
- `awkward_tangential_touch` — the boundaries meet at one point without crossing. Count 1: a touch is a point of the intersection. A tool that counted *crossings* would report 0 here and would be answering a different question.
- `awkward_overlapping_sides_infinite` — a side of P and a side of Q are collinear and overlap in a whole sub-segment, so $\partial P \cap \partial Q$ is genuinely infinite. The tool reports INFINITE and does not produce a number. This is why the theorem carries a finiteness hypothesis: without it the count is not a natural number and `ncard` would return 0.

Each configuration carries an expected count written down in advance; a mismatch, or a configuration whose polygons fail `SimpleClosed`, is reported as a discrepancy and the tool exits non-zero. It exits 0.

The 10 figures follow. In each, P is solid blue, Q is dashed red, and every point of $\partial P \cap \partial Q$ is circled and numbered.

4. Sabotage: does the checker ever say no?

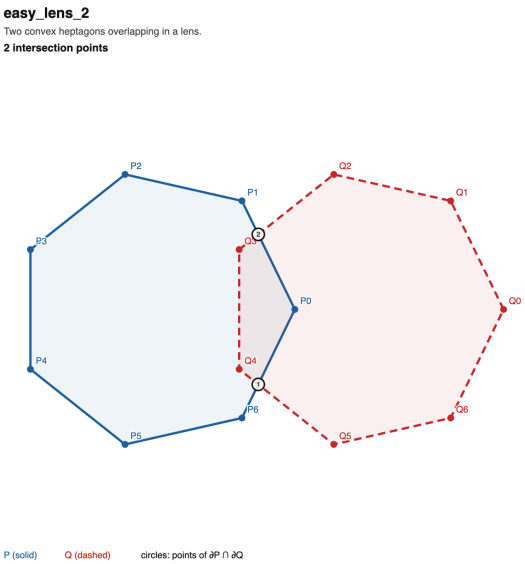
A verification that cannot fail proves nothing. If a check passes on this repository, the first question is whether it would also pass on a broken one. The repository answers this with controlled mutations:



`certificate_38`

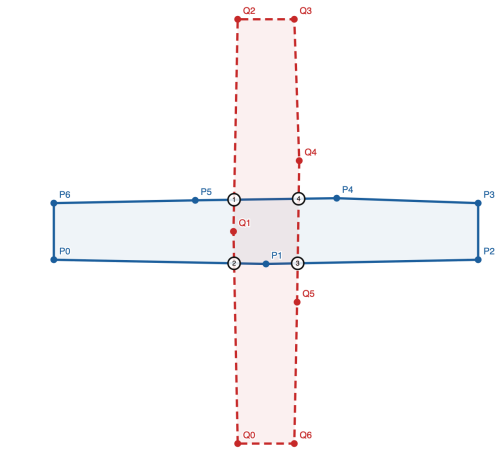
computed count: **38**

Figure 1: The lower-bound certificate. Every point of $\partial P \cap \partial Q$ is circled and numbered; there are **38**. This is the same configuration the Lean proof uses as its witness, read from the same JSON file.



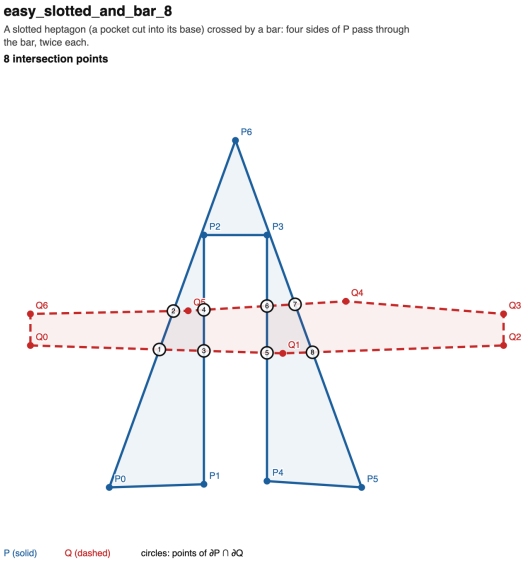
`easy_lens_2`

computed count: **2**



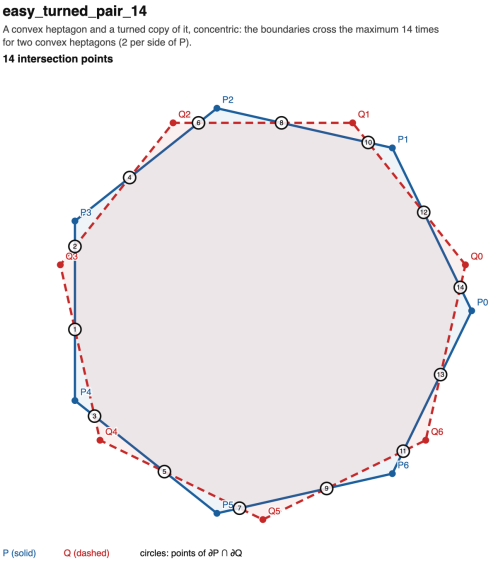
`easy_bars_cross_4`

computed count: **4**



`easy_slotted_and_bar_8`

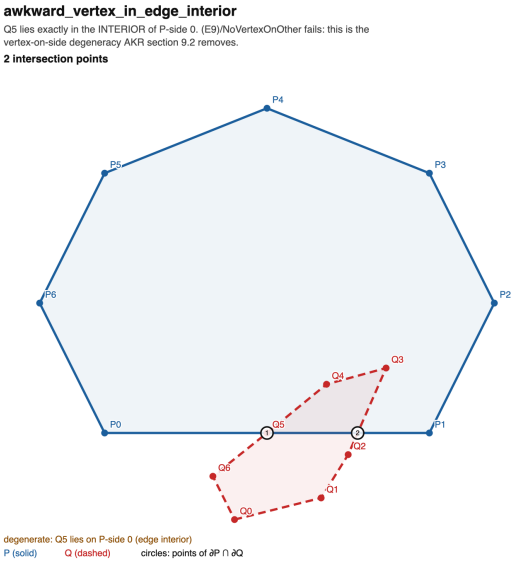
computed count: **8**



`easy_turned_pair_14`

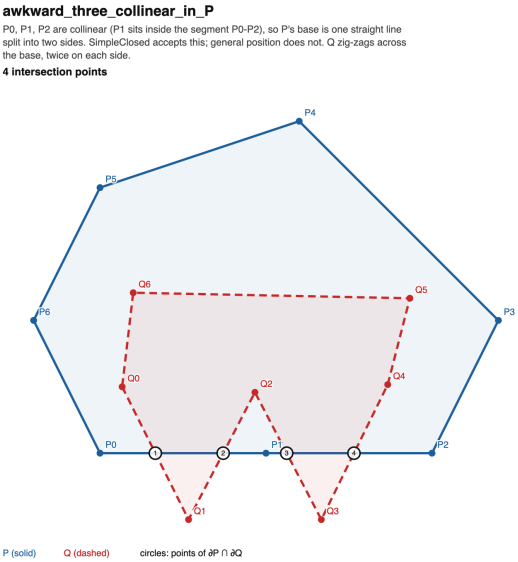
computed count: **14**

Figure 2: Sanity checks with answers you can see at a glance.



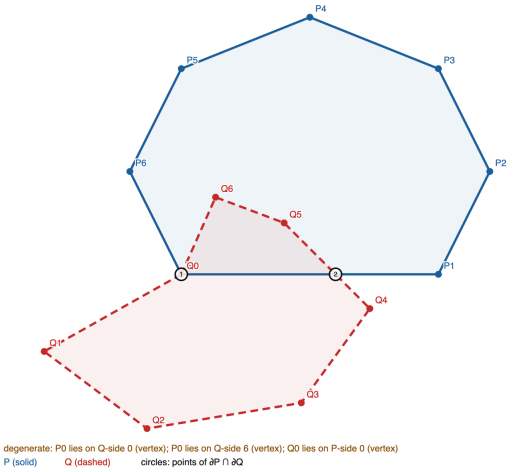
`awkward_vertex_in_edge_interior`

computed count: **2**



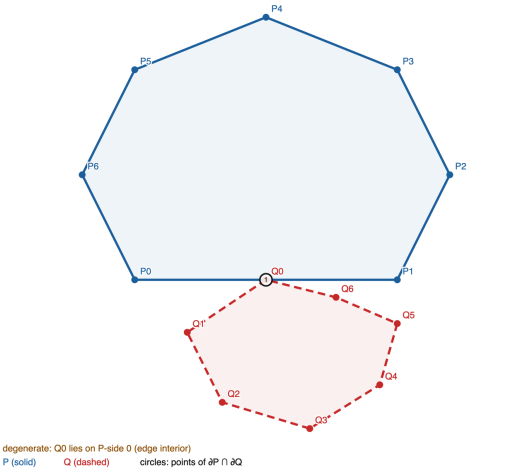
`awkward_three_collinear_in_P`

computed count: **4**



`awkward_shared_vertex`

computed count: **2**



`awkward_tangential_touch`

computed count: **1**

Figure 3: Degeneracies that the theorem's hypotheses *allow*, plus the one they exclude (`awkward_shared_vertex`).

take a throwaway copy, introduce one specific defect, and require that the check rejects it *and names the right reason*. A control copy with no defect must still be accepted, so that "rejects everything" is not mistaken for rigour.

L4 first, because it is the one that is not about broken proofs. Every other test below breaks a *proof*. L4 breaks a *definition*. The mutation changes one character in `lean/AKR9.lean`:

```
def points9 (R : Pair9) : Set (ℝ × ℝ) := bdryP9 R ∩ bdryQ9 R
```

becomes

```
def points9 (R : Pair9) : Set (ℝ × ℝ) := bdryP9 R ∪ bdryQ9 R
```

No proof text is touched. There is no `sorry`, no axiom, no `native_decide`; the mutated file is a Lean development about the union of two boundaries instead of their intersection, a set that is not finite of size at most 38. The requirement is that the build *fail*. It does: that is case L4 of the run recorded below.

The test shows that the proofs depend on `points9` at least to the extent that this change breaks them. It does not show that every unintended definition would be rejected; mutation tests demonstrate rejection of the particular mutations tested. So the 7 rows of §2 are not independent of the proofs: at least this change to them is rejected by the build.

The three groups. `scripts/negative-tests.sh` has three groups. Group N mutates the evidence files (certificates, hashes, refutation proofs). Group A mutates the *axiom log* and drives the repository's axiom checker over it — these need no Lean at all, because the checker is a log parser and can be tested the way any parser is tested. Group L is the four Lean-level mutations, L1–L4; they require a real `lake build` of each mutated copy and are opt-in behind a `--lean` flag.

Groups A and L were both run on 25 September 2026 (UTC), on the author's machine, and both transcripts are committed. The two tables below are generated from those transcripts.

One thing to know before reading them. `negative-tests.sh` can print five verdicts — *accepted*, *caught*, NOT CAUGHT, NOT APPLIED, WRONGLY REJECTED — and the generator that builds these tables renders all five, so an absence of failures below reflects the runs rather than the table. An earlier version of the generator recognised only three of the five verdicts and would have dropped a failing row; this was found and fixed before this document was built.

case	the mutation	verdict
`A1`	complete clean log (control)	accepted
	nothing — a complete, clean #print axioms log	
`A2`	wrapped axiom list (control)	accepted
	nothing — but one axiom list is wrapped across three lines	
`A3`	sorryAx in a headline theorem	**caught**
	a headline theorem's axiom list contains sorryAx	
`A4`	headline lines absent	**caught**
	a log whose headline modules were never built: clean, but they never printed	
`A5`	disallowed axiom (ofReduceBool)	**caught**
	a headline theorem depends on Lean.ofReduceBool (what native_decide leaves)	
`A6`	project-declared axiom	**caught**
	a headline theorem depends on a project-declared axiom	
`A7`	no axiom output at all	**caught**
	a build log with no #print axioms output whatsoever	
`A8`	disallowed axiom in wrapped tail	**caught**
	a disallowed axiom hidden on the wrapped continuation line	

`RESULT: 8 passed, 0 failed` (`bash scripts/negative-tests.sh --groups A`, exit 0)

Group L was run, and the transcript is committed. The four Lean-level mutations — a planted `sorry` (L1), a declared and used `axiom` (L2), a `native_decide` (L3), and the changed definition described above (L4) — each plant one defect in a copy-on-write clone of this repository, build the clone with a real `lake build`, and require the verifier to *reject* it. The run is recorded at `results/validation/2026-09-24_negative_tests_L.log`.

`date` 2026-09-25T01:37:04Z `uname` Darwin 24.6.0 arm64 `cmd` NEGTESTS_LAKE_DIR=$PWD/lean/.lake bash scripts/negative-tests.sh –lean –groups L

case	the mutation	verdict
`L1`	sorry in a dependency	**caught**
	a sorry planted three lemmas below a headline theorem; the log also shows `sorryAx`, so the mechanism under test is what fired	
`L2`	declared axiom, used	**caught**
	an axiom was declared and used beneath a headline theorem; the log also shows `points9_eq_range_ax`, so the mechanism under test is what fired	
`L3`	native_decide	**caught**
	a proof was closed by native_decide (compiler, not kernel); the log also shows `ofReduceBool/native_decide`, so the mechanism under test is what fired	
`L4`	definition changed (intersect->union)	**caught**
	points9 redefined from an INTERSECTION to a UNION (no proof touched)	

`RESULT: 4 passed, 0 failed` (`bash scripts/negative-tests.sh --lean --groups L`)

Each of the four mutations was rejected by a build of a mutated copy. This shows that these four mutations are caught; it does not show that every defect would be. L4 bears on §2 directly: no proof text was altered, only the *definition* of `points9`, and the build failed.

5. What the machine checked, and how to re-run it

working directory	*command*
`lean/`	`lake exe cache get`
`lean/`	`lake build Headline 2>&1 | tee ../headline-build.log`
repository root	`bash scripts/verify-all.sh --axiom-log headline-build.log`

The first command downloads prebuilt Mathlib files for the revision pinned in `lean/lake-manifest.json`; the second builds the headline theorems; the third is the axiom check, which needs no Lean.

`lean/Headline.lean` declares nothing. It imports the four roots of the live chain — `MainTheorem`, `RealUpperBound`, `AKR9`, `Pair9Shift` — and asks for the axiom lists of five declarations:

```
#print axioms Heptagon.MainTheorem.isGreatest_38
#print axioms Heptagon.RealUpperBound.isGreatest_38_real
#print axioms Heptagon.AKR9.isGreatest_38_of_E9
#print axioms Heptagon.Pair9Shift.isGreatest_38_of_coincFree
#print axioms Heptagon.Pair9Shift.points9_le_38_of_coincFree
```

Building that target builds the transitive closure of the four roots — 331 modules — plus `Headline` itself: 332 modules of this project, and no enumeration shard. Lake reports this as 3,344 *jobs*, a count that also includes Mathlib's modules. The last two printed declarations are the ones §2 quotes. A bare `lake build` does *not* build them: the project's `defaultTargets` omits the §9 modules, including `AKR9`, `Pair9Shift` and `Headline`.

The transcript of that command is committed. A full transcript of a local build of that target is in the repository, at `results/validation/2026-09-24_headline_build.log`: 83,020 lines, opening with a header that records the date (2026-09-25T01:36:38Z), the machine, the Lean version, and the command. The header does not record a commit. The Lean sources were last changed in commit `a211140` (24 September 2026, 23:59 UTC), before this build, and have not changed since; the transcript was committed in `b070c2f`. Read out of that file:

- it ends with `Build completed successfully (3,344 jobs).`;
- it carries 408 `depends on axioms` lines and one line for a declaration that depends on no axioms — 409 declarations in all — and across them exactly 3 distinct axioms appear: `propext`, `Classical.choice` and `Quot.sound`. There is no fourth entry anywhere in the file;
- `sorryAx` occurs 0 times in it, and `Lean.ofReduceBool` — what `native_decide` would leave — 0 times;
- each of the six theorems the repository's checker demands printed an axiom line of its own. Five of the `#print axioms` *directives* that produce those lines sit in `lean/Headline.lean` at lines 45–49. The sixth, for `points_le_38_of_no_vertex_on_boundary`, is a directive at `lean/AKR9.lean:420` — the *theorem* itself is stated at `lean/AKR9.lean:354` — which is why that one axiom line appears about a thousand lines earlier in the log than the other five.

The repository's axiom checker is a log parser that needs no Lean toolchain, and it can be pointed at that file directly:

```
bash scripts/verify-all.sh --axiom-log
results/validation/2026-09-24_headline_build.log
```

It exits 0, and reports 409 declarations parsed, 0 of them using a disallowed axiom, and 6 of 6 headline theorems confirmed. Its full output is committed at `docs/audit/out/axiom_check_headline.log`. The transcript's own provenance header, its six headline axiom lines, its last line, and the checker's verdict on it are reproduced here:

```
# date  : 2026-09-25T01:36:38Z
# uname : Darwin 24.6.0 arm64
# lean  : Lean (version 4.33.1, arm64-apple-darwin24.6.0,
```

```
           commit 819816b2e0a3bf405af45ae5c7af2491d8f5bee6, Release)
# cmd   : cd lean && lake build Headline
  …
info: AKR9.lean:420:0: 'Heptagon.AKR9.points_le_38_of_no_vertex_on_boundary'
    depends on axioms: [propext, Classical.choice, Quot.sound]
info: Headline.lean:45:0: 'Heptagon.MainTheorem.isGreatest_38'
    depends on axioms: [propext, Quot.sound]
info: Headline.lean:46:0: 'Heptagon.RealUpperBound.isGreatest_38_real'
    depends on axioms: [propext, Classical.choice, Quot.sound]
info: Headline.lean:47:0: 'Heptagon.AKR9.isGreatest_38_of_E9'
    depends on axioms: [propext, Classical.choice, Quot.sound]
info: Headline.lean:48:0: 'Heptagon.Pair9Shift.isGreatest_38_of_coincFree'
    depends on axioms: [propext, Classical.choice, Quot.sound]
info: Headline.lean:49:0: 'Heptagon.Pair9Shift.points9_le_38_of_coincFree'
    depends on axioms: [propext, Classical.choice, Quot.sound]
Build completed successfully (3344 jobs).
```

Quoted from `results/validation/2026-09-24_headline_build.log` (83,020 lines): the provenance header, the six headline axiom lines (log lines 82448, 83015, 83016, 83017, 83018, 83019), and the final line 83020.

The repository's own axiom checker, run over that transcript (the command boxed above; exit 0) reports:

```
'depends on axioms' lines found          : 408
declarations parsed                      : 409
declarations using a disallowed axiom    : 0
occurrences of sorryAx                   : 0
distinct axioms                          : 3
headline theorems confirmed              : 6 of 6
```

AXIOM CHECK: CLEAN — 409 declaration(s) reported their axioms; every one of the 3 distinct axioms used is among propext, Classical.choice, Quot.sound; sorryAx appears nowhere; all 6 headline theorems confirmed.

Six axiom lines are not six unconditional theorems. They all carry the number 38 and they are not the same claim, so they are worth separating before you read the list above.

Five of the six are statements about point sets in $\mathbb{R}^2$, and they differ in their hypotheses:

- `points9_le_38_of_coincFree` and `isGreatest_38_of_coincFree`: finiteness and no shared vertex — the statement of §2;
- `points_le_38_of_no_vertex_on_boundary` and `isGreatest_38_of_E9`: the older, stronger hypothesis (E9), that no vertex of either polygon lies anywhere on the other's boundary; finiteness is then a consequence. Superseded by the two above;
- `isGreatest_38_real`: general position, carried as a field of the structure `RConfig`.

Each assumes only the hypotheses written into its own statement. The crossing-pair bound they rest on, `RawConfig.raw_crossPairs_card_le_38` (`lean/RawConfig.lean:231`), takes a pair of point-set simple heptagons with no general-position hypothesis.

The sixth is a different animal. `MainTheorem.isGreatest_38` (`lean/MainTheorem.lean:88`) is about a different object — integer-coordinate, orientation-level `Config`s — and it carries three explicit hypotheses which are what that route does not itself prove: that every configuration has an even crossing count, that none attains 42, and that none attains 40, the last of these being the entire 13,003,620-graph enumeration. Its own doc-comment (`lean/MainTheorem.lean:45--48`) puts it plainly:

"So the honest reading of `isGreatest_38` below is: '38 is the maximum crossing count over integer-coordinate, orientation-level heptagon configurations, GIVEN the listed hypotheses.' Getting from there to `f(7,7) = 38` needs the Jordan-curve step and the general-position reduction, both external."

Note in the transcript above that this one has the *shortest* axiom list of the six — `[propext, Quot.sound]`, with no `Classical.choice`. That is a consequence of its being conditional, not a sign that it is the strongest of them: an undischarged hypothesis costs no axioms. Nothing in this document rests on it. The claim in §1, the statement in §2 and the non-vacuity of §7 are all the other route.

Two figures in this section that the transcript does *not* establish. It records no elapsed time, so the **11 s** warm-build figure is still only the repository's own assertion (`lean/README.md`) and you should read it as that. And the log counts *jobs*, not modules: the 3,344 above is a job count, while the module count (331 in the closure of the four roots, 332 with `Headline`) comes from a transitive-closure computation over the `import` lines, repeated for this revision.

A correction, and what the axiom check really guards against. An earlier draft of this section asserted that `#print axioms` prints only when a module is actually recompiled, so that a warm rebuild "replays nothing and therefore prints nothing" and only a cold build could produce this evidence. **That was wrong**, and the committed transcript is what demonstrates it: all 271 of its progress lines read `Replayed` and not one reads `Built` — it is a fully warm run — and it nevertheless carries every one of the 408 axiom lines, because Lake replays the cached messages of the modules it does not recompile.

The trap the checker really guards against is a different one, and a worse one: a log in which the headline theorems never appear *at all*. That is what a bare `lake build` produces here, because `defaultTargets` omits the §9 modules, so those theorems are never part of the build and never print. Such a log exits 0 and looks clean while establishing nothing whatever about the statements you care about. Test A4 in §4 is exactly that case and the checker rejects it. Its requirement is not that the build was cold; it is that each of the six named theorems printed an axiom line of its own.

Cost of a first build. A fresh clone must first download Mathlib (the Lean mathematical library, pinned here at tag `v4.33.1`) before any of this project's own code compiles. The repository records about 1.4 GB in `lean/.lake` after a cold build, measured before later modules were added, so treat it as a lower bound. On the GitHub-hosted runners of §6 (2 cores) the headline build step took about 32 minutes in the second run.

6. Recorded CI builds

The evidence in §§2–5 was produced on the author's computer. The repository also has a GitHub Actions workflow (`.github/workflows/verify.yml`) that, on each push to the main branch, checks out the repository on a GitHub-hosted runner, installs the pinned Lean toolchain, fetches prebuilt Mathlib files, builds the `Headline` target, and runs the axiom check and the group-A tests of §4. Two runs are recorded, both passing:

run	*commit*	*started (UTC), duration*
36086080499	b78f372	25 Sep 2026 02:23, 40 min 54 s
36089101301	e984888	25 Sep 2026 03:07, 35 min 05 s

Both on `ubuntu-latest`, 2 cores, 7.8 GiB. Both commits have the Lean sources of `a211140`. Run pages are at `github.com/NewYorkImperialist/heptagon-intersections/actions/runs/<run>`.

What the first run reported, quoted from the extract committed at `docs/audit/out/ci_run.log`:

```
lean/lean-toolchain pins: leanprover/lean4:v4.33.1
Build completed successfully (3344 jobs).
OK: 0 occurrences of sorryAx
   axiom-check: declarations parsed              : 409
   axiom-check: distinct axioms                  : 3
   axiom-check: headline theorems confirmed      : 6 of 6
   AXIOM CHECK: CLEAN
RESULT: 8 passed, 0 failed          (the group-A tests)
```

The second run reported the same figures; its extract is `docs/audit/out/ci_run_2.log`. They agree with the local transcript of §5: the same job count, the same 409 declarations, the same six theorems and the same three axioms.

The project's modules were compiled, not restored. Lake can restore a previous build and replay its stored messages, which look the same as freshly produced ones. The workflow does not cache `lean/.lake/build`. In each run, Lake's progress output has 346 lines marked *Built* and none marked *Replayed*: 332 are this project's modules (the whole `Headline` closure) and 14 are jobs building Mathlib's `cache` tool. Mathlib itself is fetched prebuilt for the pinned revision.

What these runs do and do not show. They record that the committed workflow passed at those two revisions on a machine not controlled by the author. The workflow itself is written by the repository owner, so the runs show what that workflow checks, and they do not cover later changes to the repository. They do not run the Lean mutation tests L1–L4 of §4 (an opt-in job, not run; those results come from the local transcript), and they build the 332-module headline closure, not all 1,779 declared libraries. They are not a review of the mathematics or of the formal statements.

Access. As of 4 October 2026 the repository is private (checked with the GitHub command-line tool). The run pages and their artifacts are visible only to people with access to the repository, and the artifacts expire 90 days after each run.

7. Non-vacuity: the hypotheses can be met, and the bound is attained

A conditional theorem whose hypotheses are contradictory is vacuously true and says nothing. An upper bound nobody attains is not a maximum. Both failure modes are closed explicitly at the end of `lean/Pair9Shift.lean`, by a single bundled declaration, `nonVacuity`, in the `Heptagon.Pair9Shift` namespace.

`lean/Pair9Shift.lean:5296`

```
theorem nonVacuity :
    Nonempty AKR9.Pair9
    ∧ (points9 AKR9.certPair9).Nonempty
    ∧ (points9 AKR9.certPair9).Finite
    ∧ (∀ v j : Fin 7, AKR9.certPair9.V (pP v) ≠ AKR9.certPair9.V (pQ j))
    ∧ AKR9.NoVertexOnOther AKR9.certPair9
    ∧ (points9 AKR9.certPair9).ncard = 38
    ∧ ¬ (points9 AKR9.certPair9).ncard ≤ 37
    ∧ (38 : ℕ) ∈ attainedCoincFree
    ∧ (39 : ℕ) ∉ attainedCoincFree
    ∧ IsGreatest attainedCoincFree 38 :=
  ⟨pair9_nonempty, certPair9_points9_nonempty, certPair9_finite,
   certPair9_coincFree, certPair9_noVertexOnOther, AKR9.certPair9_ncard,
   certPair9_not_le_37, mem_attainedCoincFree_38, not_mem_attainedCoincFree_39,
   isGreatest_38_of_coincFree⟩
```

Ten clauses, in order:

(1) `Pair9` is inhabited: the two `SimpleClosed` hypotheses of §2 are satisfiable at all. Without this the whole theory could be about an empty type.

(2) The witness's intersection set is non-empty.

(3) It is *finite* — so `ncard` is the real cardinality. On an infinite set `ncard` returns 0, which would make the bound ≤ 38 true and meaningless; this clause closes that door. (Clause (6) already implies it, for exactly that reason; it is stated separately for the reader.)

(4) The witness satisfies `hcoinc`: it shares no vertex. The hypothesis of the theorem is not contradictory.

(5) The witness also satisfies the *stronger* condition `NoVertexOnOther` — no vertex anywhere on the other boundary.

(6) Its count is exactly 38: **the bound is attained**.

(7) ≤ 37 is *false* there: **the bound is not slack**. One less is provably wrong, so 38 is the right constant and not an artefact of a lossy estimate. (Immediate from (6); stated because it is the thing a reader wants to see said.)

(8) 38 is an attained value.

(9) 39 is *not* attained.

(10) 38 is the greatest attained value.

These are ten clauses, not ten logically independent facts: (6) is the sharpest of them and already implies (3) and (7). The content is not the number of clauses but the fact that **(2)–(7) all hold of the *same* configuration**, `AKR9.certPair9`. Checking hypotheses one at a time would not rule out a theorem whose hypotheses are each satisfiable but jointly contradictory; a single shared witness does.

A companion records that the maximum is *unique* — any n that is a greatest attained value equals 38:

`lean/Pair9Shift.lean:5272`

```
theorem isGreatest_attainedCoincFree_unique {n : ℕ}
    (h : IsGreatest attainedCoincFree n) : n = 38 :=
  h.unique isGreatest_38_of_coincFree
```

8. What is not proved

$f(7,7) = 38$ is not a Lean theorem. What is kernel-checked is the statement in §2: the bound 38 for simple heptagons that share no vertex and whose boundaries meet in finitely many points, with 38 attained. Finiteness is a hypothesis, not a conclusion; the last row of §3 is a configuration where it fails. The doc-comment on the theorem says the same: "*This is not `f(7,7) = 38`, and it is not all of AKR §9.*"

The vertex-coincidence reduction. The reference is E. Ackerman, B. Keszegh and G. Rote, *An almost optimal bound on the number of intersections of two simple polygons*, Discrete Comput. Geom. **68** (2022) 1049–1077; §9 appears only in this journal version. Their §9 shows that degenerate cases — a vertex of one polygon on the boundary of the other — can be removed without decreasing the number of intersection points. It has two parts. §9.2's part, a vertex in the *interior* of a side, is *proved here* in Lean, by the project's own argument; that is what `points9_le_38_of_coincFree` adds over its (E9) predecessor. §9.1's part, a vertex *equal* to a vertex of the other polygon, is not proved here. To obtain $f(7,7) = 38$ one needs:

(a) *kernel-checked:* the statement of §2;

(b) *from an external theorem:* the reduction of AKR §9.1, that every pair can be replaced by one sharing no vertex without decreasing the number of intersection points;

(c) *not settled in this project:* that the replaced pair is again simple in the sense of §2 and again has finite intersection, as §2's hypotheses require. We have not found either stated explicitly in AKR §9. The project has a kernel-checked lemma that simplicity survives every sufficiently small linear motion of the vertices (`SimplicityStable.heptagon_simplicity_stable_linear`), and unreviewed pen-and-paper notes on choosing the §9.1 directions so that the intersection stays finite, but it has not assembled these with §9.1 into a proof of the reduction for these definitions.

Using a published theorem does not by itself make an argument incomplete; the qualification here is item (c).

Points the project noted in AKR §9. Two points arose while working on §9, recorded here because they bear on how much of §9 can be cited unchanged. They concern the procedure as printed, not the truth of the theorem. First, §9.2 moves the endpoints of a side along the lines of the adjacent sides; when two consecutive sides are collinear, a configuration the definitions of §2 allow, the printed description does not determine the move, and the project uses a different ("tilted") shift. Second, the criterion stated in §9.1 for choosing perturbation directions does not, as written, exclude directions that make the perturbed intersection infinite; an additional condition, which AKR may regard as implicit, is needed. The project's supplements are its own and are not reviewed. These points are documented in `docs/verification-status.md`, rows 60–62.

Review status. No independent review of the mathematics, the Lean development or the Python tools has been completed. The project's records (`docs/verification-status.md`, row 18) note that a colleague began reading an earlier dependency manifest in September 2026, and record no findings from that reading. Human review still matters for the statement and definitions (§2), for the external reduction and its applicability (above), and for the connection between this document and the code that was checked (§§2 and 5).

The gallery. The gallery in §3 is a Python *transcription* of the Lean predicates, checked against them by hand and annotated with line numbers. It is evidence that the definitions compute what one expects on ten examples, not a proof that the Python and the Lean agree.

Earlier gaps, now closed. An earlier draft listed two further items. That the §5 build figures had no committed transcript is no longer true: the transcript is at `results/validation/2026-09-24_headline_build.log`, and the axiom checker can be run over it without Lean. That group L of the mutation suite had never been run in a recorded way is no longer true either: it was run, its transcript is committed, and §4 reports each case.

A. The statement surface, verbatim and complete

All 16 declarations extracted by `docs/audit/extract_lean.py`, unedited, with file and line. The six that carry the content appear in §2; these are the rest, and the full text of those six.

pP — lean/ResidualSymBase.lean:23

```
def pP (i : Fin 7) : Fin 14 := ⟨i.val, by omega⟩
```

pQ — lean/ResidualSymBase.lean:26

```
def pQ (i : Fin 7) : Fin 14 := ⟨7 + i.val, by omega⟩
```

SimpleClosed — lean/AKR9.lean:71

```
def SimpleClosed (W : Fin 7 → ℝ × ℝ) : Prop :=
  (∀ i : Fin 7, W i ≠ W (i + 1)) ∧
  (∀ i j : Fin 7, ¬ LowerBound.Adjacent i j →
      Disjoint (segment ℝ (W i) (W (i + 1))) (segment ℝ (W j) (W (j + 1)))) ∧
  (∀ i : Fin 7,
      segment ℝ (W i) (W (i + 1)) ∩ segment ℝ (W (i + 1)) (W (i + 2))
        = {W (i + 1)})
```

Pair9 — lean/AKR9.lean:81

```
structure Pair9 where
  /-- The fourteen vertices: `P` is `0..6`, `Q` is `7..13`. -/
  V : Fin 14 → ℝ × ℝ
  simpleP : SimpleClosed (fun i : Fin 7 => V (pP i))
  simpleQ : SimpleClosed (fun i : Fin 7 => V (pQ i))
```

NoVertexOnOther — lean/AKR9.lean:103

```
def NoVertexOnOther (R : Pair9) : Prop :=
  (∀ i j : Fin 7, R.V (pP i) ∉ segment ℝ (R.V (pQ j)) (R.V (pQ (j + 1)))) ∧
  (∀ i j : Fin 7, R.V (pQ j) ∉ segment ℝ (R.V (pP i)) (R.V (pP (i + 1))))
```

sideP9 — lean/AKR9.lean:109

```
def sideP9 (R : Pair9) (i : Fin 7) : Set (ℝ × ℝ) :=
  segment ℝ (R.V (pP i)) (R.V (pP (i + 1)))
```

sideQ9 — lean/AKR9.lean:112

```
def sideQ9 (R : Pair9) (j : Fin 7) : Set (ℝ × ℝ) :=
  segment ℝ (R.V (pQ j)) (R.V (pQ (j + 1)))
```

bdryP9 — lean/AKR9.lean:115

```
def bdryP9 (R : Pair9) : Set (ℝ × ℝ) := ⋃ i : Fin 7, sideP9 R i
```

bdryQ9 lean/AKR9.lean:116

```
def bdryQ9 (R : Pair9) : Set (ℝ × ℝ) := ⋃ j : Fin 7, sideQ9 R j
```

points9 lean/AKR9.lean:119

```
def points9 (R : Pair9) : Set (ℝ × ℝ) := bdryP9 R ∩ bdryQ9 R
```

attainedCoincFree lean/Pair9Shift.lean:4314

```
def attainedCoincFree : Set ℕ :=
  { n | ∃ R : Pair9, (points9 R).Finite ∧
      (∀ v j : Fin 7, R.V (pP v) ≠ R.V (pQ j)) ∧ (points9 R).ncard = n }
```

points9_le_38_of_coincFree lean/Pair9Shift.lean:4223

```
theorem points9_le_38_of_coincFree (R : Pair9) (hfin : (points9 R).Finite)
    (hcoinc : ∀ v j : Fin 7, R.V (pP v) ≠ R.V (pQ j)) :
    (points9 R).ncard ≤ 38 := by
  obtain ⟨Rend, hD, hE, hb⟩ := exists_chain_deg_free _ R hfin hcoinc rfl
  exact points9_le_38_of_joint R Rend hfin hb hD hE
```

isGreatest_38_of_coincFree lean/Pair9Shift.lean:4324

```
theorem isGreatest_38_of_coincFree : IsGreatest attainedCoincFree 38 := by
  refine ⟨⟨AKR9.certPair9, certPair9_finite, certPair9_coincFree,
    AKR9.certPair9_ncard⟩, ?_⟩
  rintro n ⟨R, hfin, hco, rfl⟩
  exact points9_le_38_of_coincFree R hfin hco
```

nonVacuity lean/Pair9Shift.lean:5296

```
theorem nonVacuity :
    Nonempty AKR9.Pair9
    ∧ (points9 AKR9.certPair9).Nonempty
    ∧ (points9 AKR9.certPair9).Finite
    ∧ (∀ v j : Fin 7, AKR9.certPair9.V (pP v) ≠ AKR9.certPair9.V (pQ j))
    ∧ AKR9.NoVertexOnOther AKR9.certPair9
    ∧ (points9 AKR9.certPair9).ncard = 38
    ∧ ¬ (points9 AKR9.certPair9).ncard ≤ 37
    ∧ (38 : ℕ) ∈ attainedCoincFree
    ∧ (39 : ℕ) ∉ attainedCoincFree
    ∧ IsGreatest attainedCoincFree 38 :=
  ⟨pair9_nonempty, certPair9_points9_nonempty, certPair9_finite,
   certPair9_coincFree, certPair9_noVertexOnOther, AKR9.certPair9_ncard,
   certPair9_not_le_37, mem_attainedCoincFree_38, not_mem_attainedCoincFree_39,
   isGreatest_38_of_coincFree⟩
```

certPair9_not_le_37 lean/Pair9Shift.lean:5252

```
theorem certPair9_not_le_37 : ¬ (points9 AKR9.certPair9).ncard ≤ 37 := by
  intro h
  rw [AKR9.certPair9_ncard] at h
  omega
```

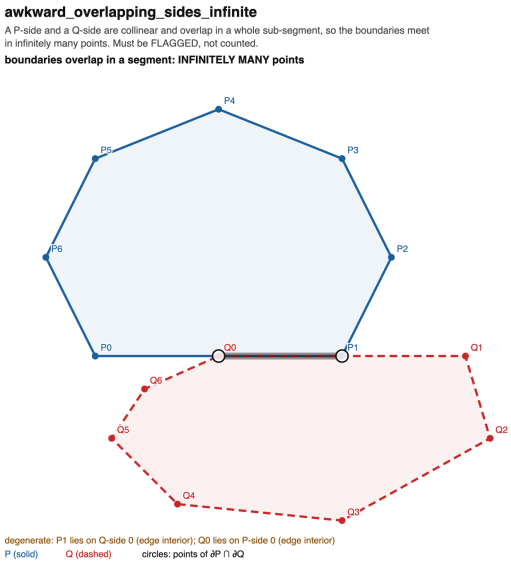
isGreatest_attainedCoincFree_unique lean/Pair9Shift.lean:5272

```
theorem isGreatest_attainedCoincFree_unique {n : ℕ}
    (h : IsGreatest attainedCoincFree n) : n = 38 :=
  h.unique isGreatest_38_of_coincFree
```

B. How this document was produced

```
python3 docs/audit/extract_lean.py    # verbatim Lean   -> out/lean_excerpts.*
python3 docs/audit/gallery.py         # figures, counts -> out/gallery/
for f in docs/audit/out/gallery/*.svg; do            # SVG -> PNG (macOS)
  qlmanage -t -s 1400 -o docs/audit/out/png "$f"; done
bash scripts/negative-tests.sh --groups A \
     > docs/audit/out/negative_tests_A.log           # group A table in 4
bash scripts/negative-tests.sh --lean --groups L \                # group L in 4
     > results/validation/2026-09-24_negative_tests_L.log
bash scripts/verify-all.sh \                                      # the numbers in 5
     --axiom-log results/validation/2026-09-24_headline_build.log \
     > docs/audit/out/axiom_check_headline.log
gh run view 36089101301 --log                     # source of out/ci_run_2.log
python3 docs/audit/make_tables.py     # the LaTeX fragments
python3 docs/audit/extract_lean.py --check   # drift tripwire; MUST exit 0
cd docs/audit && tectonic audit.tex
```

The Lean text in §2, §7 and Appendix A is extracted by `extract_lean.py`, and the counts are generated by the scripts above. The prose in the left column of §2 and in the notes beneath each row is written by hand, and it is what a reader is asked to check against the Lean.



`awkward_overlapping_sides_infinite`

computed count: **infinite**

Figure 4: Excluded by the finiteness hypothesis: two sides overlap in a whole sub-segment, so the intersection is genuinely infinite. It is flagged, not miscounted.